TRADINGLAB 2.0 ? DOCUMENTO DE ARQUITECTURA

TradingLab 2.0 — Workflow de TradingLab MT5 Runner

Distribución y operación segura entre VPS mediante Google Drive

Versión: 1.0
Estado: Propuesta formal incorporada a Arquitectura 2.2
Fecha: 7 de octubre de 2026
Canal inicial: Google Drive
Documentos relacionados: `tradinglab-architecture-v2.md`, `sqx-mt5-incubation-pipeline.md`

1. Propósito

Este documento define cómo crear, revisar, empaquetar, compartir, instalar, ejecutar y actualizar `TradingLab MT5 Runner`. También especifica cómo intercambiar jobs y evidencias entre la VPS TradingLab y la VPS MT5 sin habilitar acceso remoto entre ellas.

Google Drive será el canal de intercambio inicial. Drive no será fuente de verdad, sistema de autorización ni mecanismo de ejecución automática. Los paquetes autoritativos se conservarán en storage y catálogo de TradingLab.

2. Situación operativa

Existen dos zonas separadas:

Zona TradingLab

- Ejecuta SQX.
- Conserva datos y documentación.
- Prepara estrategias y solicitudes de trabajo.
- Mantiene Git, catálogo y storage.
- Compara resultados SQX/MT5.
- No tiene acceso a la VPS MT5.

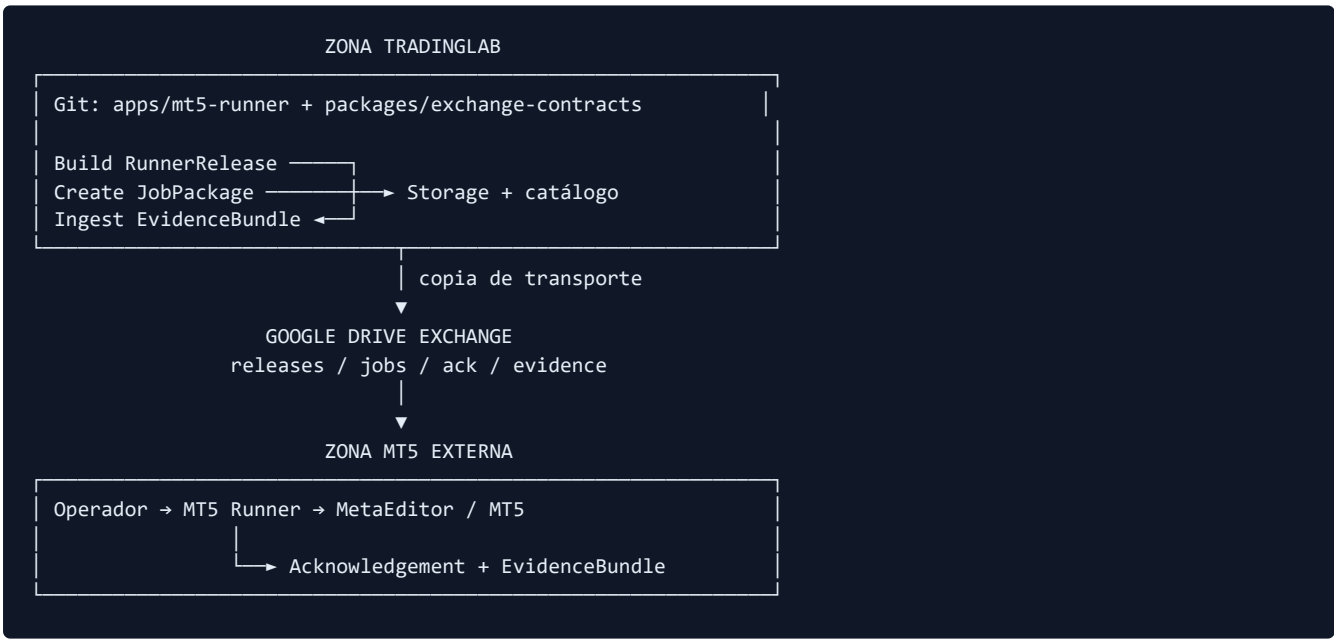
Zona MT5 externa

- Es administrada por el socio.
- Ejecuta MetaEditor y MT5.
- Compila cuando corresponda.
- Ejecuta backtests e incubaciones.
- Puede ejecutar scripts publicados por TradingLab.
- No expone acceso remoto a TradingLab.

3. Objetivos de seguridad

1. No abrir conexiones entrantes desde TradingLab hacia MT5.
2. No almacenar credenciales de la VPS MT5 en TradingLab.
3. No entregar acceso al repositorio completo a la VPS MT5 durante la fase inicial.
4. No ejecutar automáticamente archivos por aparecer en Drive.
5. No aceptar comandos arbitrarios dentro de un job.
6. Verificar integridad y versión de todo paquete.
7. Mantener aprobación local para acciones sensibles.
8. Producir evidencia suficiente para reconciliar lo solicitado con lo ejecutado.
9. Permitir rollback del runner y de deployments.
10. Preparar una evolución hacia mayor automatización sin debilitar la frontera.

4. Arquitectura



La VPS MT5 realiza pull de información o el socio descarga manualmente. TradingLab nunca hace push remoto dentro de esa VPS.

5. Código fuente

El runner vive dentro del monorepo:

```
apps/
└── mt5-runner/
    ├── src/
    ├── commands/
    ├── schemas/
    ├── scripts/
    ├── tests/
    ├── install.ps1
    └── README.md

packages/
├── bot-pipeline/
└── exchange-contracts/
```

- `apps/mt5-runner/` : aplicación local para Windows/MT5.
- `packages/bot-pipeline/` : preparación, compilación, gates y empaquetado.
- `packages/exchange-contracts/` : esquemas de release, job, acknowledgement y evidencia.

El runner se desarrolla en `main`. No se creará una rama permanente para la VPS MT5: una rama representa desarrollo, no una versión instalada ni una frontera de seguridad.

6. Artefactos del intercambio

`RunnerRelease`

Paquete versionado del software que se instala en la VPS MT5.

```
tradinglab-mt5-runner-1.0.0.zip
├── runner/
├── scripts/
├── schemas/
├── VERSION
├── release-manifest.json
└── checksums.sha256
```

`JobPackage`

Solicitud declarativa para compilar, probar, desplegar o incubar.

```
job-<job-id>.zip
├── job-manifest.yaml
├── checksums.sha256
├── source/
├── includes/
├── preset.set
└── instructions.md
```

`Acknowledgement`

Mensaje estructurado que confirma recepción, aprobación, rechazo, inicio, finalización o fallo del job.

`EvidenceBundle`

Resultado devuelto por la VPS MT5.

```
evidence-<job-id>.zip
├── evidence-manifest.yaml
├── checksums.sha256
├── compiler.log
├── mt5-report.html
├── tester.log
├── effective-preset.set
├── incubation-events.json
└── screenshots/
```

7. Estructura de Google Drive

```
TradingLab Exchange/
├── tradinglab-to-mt5/
│   ├── runner-releases/
│   └── jobs/
│
└── mt5-to-tradinglab/
    ├── acknowledgements/
    └── evidence/
```

Reglas

- Cada paquete tiene identificador y nombre único.
- Los paquetes publicados no se sobrescriben.
- Una corrección produce otra versión o job.
- Los permisos se limitan a las carpetas necesarias.
- Los archivos recibidos se copian a un inbox local antes de procesarse.
- Drive puede limpiarse según política una vez confirmada la persistencia autoritativa.
- La ruta o fecha de Drive no define identidad; la identidad proviene del ID y los hashes.

8. Instalación en la VPS MT5

Estructura local sugerida:

```
C:\TradingLabMT5\
├── runner\
│   ├── versions\
│   │   ├── 1.0.0\
│   │   └── 1.1.0\
│   └── current\
├── exchange\
│   ├── inbox\
│   ├── accepted\
│   ├── rejected\
│   └── evidence\
├── workspaces\
├── logs\
└── config\
```

El directorio `current` apunta lógica u operacionalmente a la versión activa. La implementación exacta deberá evitar enlaces que compliquen permisos o herramientas de Windows; puede mantenerse mediante archivo de configuración o copia controlada.

9. Workflow A — Crear un RunnerRelease

A1. Preparar la versión

1. Seleccionar un commit limpio de `main`.
2. Actualizar versión y changelog.

3. Confirmar compatibilidad con los contratos de exchange.
4. Ejecutar tests unitarios, integración y pruebas con fixtures MT5.
5. Revisar cambios de seguridad y permisos.

A2. Construir

1. Crear un workspace limpio.
2. Copiar solo runtime, scripts y esquemas necesarios.
3. Excluir código, documentos y secretos ajenos al runner.
4. Generar `release-manifest.json`.
5. Calcular SHA-256 de cada archivo y del paquete final.
6. Crear el ZIP versionado.

A3. Revisar y aprobar

1. Verificar que el paquete no incluya credenciales.
2. Revisar lista de archivos.
3. Instalar en entorno de prueba limpio.
4. Ejecutar self-test.
5. Aprobar o rechazar la versión.

A4. Publicar

1. Guardar el original en storage.
2. Registrar versión, commit, hashes y aprobación en catálogo.
3. Copiarlo a `tradinglab-to-mt5/runner-releases/`.
4. Notificar al operador mediante el canal acordado.

10. Workflow B — Instalar o actualizar el runner

B1. Descargar

El operador descarga manualmente el paquete desde Google Drive a un directorio de staging. No se ejecuta desde la carpeta sincronizada.

B2. Verificar

En PowerShell, el operador puede calcular inicialmente:

```
Get-FileHash .\tradinglab-mt5-runner-1.0.0.zip -Algorithm SHA256
```

El resultado debe coincidir con el hash comunicado por un canal confiable o contenido en un manifiesto autenticado según la política vigente.

También se verifica:

- nombre y versión;
- tamaño razonable;
- lista de archivos esperada;
- ausencia de rutas absolutas o traversal;

- compatibilidad del sistema;
- aprobación registrada.

B3. Instalar

1. Detener tareas del runner, no necesariamente MT5 si no corresponde.
2. Extraer a `runner\versions\<version>`.
3. Ejecutar self-test sin acciones operativas.
4. Configurar `current` para la nueva versión.
5. Registrar usuario, fecha y hash instalado.
6. Conservar la versión anterior.

B4. Rollback

Si falla:

1. Detener la nueva versión.
2. Reactivar la versión anterior.
3. Conservar logs.
4. Emitir un acknowledgement de rollback.
5. No modificar paquetes ni reutilizar el mismo número de versión.

11. Configuración local del runner

La configuración contiene rutas y alias, nunca secretos en texto dentro del repositorio o paquete.

Ejemplo conceptual:

```
runner_id: mt5-vps-01
environment_id: env_01...
mt5_terminal_alias: incubation-terminal
allowed_symbols:
  - XAUUSD
allowed_timeframes:
  - M5
  - H4
allow_dll: false
allow_network_from_ea: false
require_local_approval:
  - compile
  - deploy
  - incubation_start
  - incubation_stop
```

Las credenciales de Drive, Windows, broker o cuenta MT5 se administran mediante mecanismos locales seguros y no aparecen en evidencias.

12. Workflow C — Crear un JobPackage

C1. Definir el trabajo

TradingLab crea un job con:

- identidad y versión de esquema;
- runner mínimo compatible;
- `StrategyVersion` y `BotBuild` esperado;
- acciones permitidas;
- artefactos de entrada y hashes;
- símbolo, timeframe y periodo;
- límites de recursos y tiempo;
- controles de seguridad;
- outputs obligatorios;
- expiración.

Ejemplo:

```
schema_version: 1
job_id: job_01...
job_type: mt5-backtest
required_runner_version: ">=1.0.0 <2.0.0"
expires_at: "2026-10-14T18:00:00Z"

strategy_version_id: stv_01...
requested_build_id: bld_01...

actions:
  - compile
  - smoke_test
  - backtest
  - collect_evidence

inputs:
  source_sha256: "..."
  preset_sha256: "..."

constraints:
  symbol: XAUUSD
  timeframe: M5
  max_duration_minutes: 120
  allow_dll: false
  allow_network: false
```

No existe un campo `command`, `powershell` o equivalente que permita ejecución arbitraria.

C2. Empaquetar

1. Materializar inputs en workspace limpio.
2. Validar contra el esquema.
3. Generar checksums.
4. Crear paquete inmutable.
5. Guardarlo en storage y catálogo.

C3. Publicar

1. Copiar a `tradinglab-to-mt5/jobs/`.
2. Registrar `published_at`.
3. Notificar al operador.
4. No considerar el job recibido hasta obtener acknowledgement.

13. Workflow D — Recibir y aprobar un job

D1. Ingreso

1. Descargar o copiar desde Drive a `exchange\inbox`.
2. Abrir el paquete mediante una rutina segura.
3. Rechazar rutas absolutas, traversal y archivos inesperados.
4. Verificar tamaño total y límites de descompresión.
5. Validar hashes y esquema.

D2. Evaluación

El runner verifica:

- versión mínima/máxima compatible;
- expiración;
- `job_id` no procesado;
- acciones incluidas en allowlist;
- símbolos y timeframes permitidos;
- política de DLL/red;
- inputs completos;
- espacio y herramientas disponibles.

D3. Aprobación

El runner muestra un plan legible:

```
Job: job_01...
Acciones: compile, smoke_test, backtest, collect_evidence
Símbolo: XAUUSD
Timeframe: M5
Duración máxima: 120 minutos
DLL: no
Red desde EA: no
```

El operador aprueba o rechaza. Se genera un acknowledgement firmado o al menos hasheado y relacionado con el job.

14. Workflow E — Ejecutar el job

Preparación

1. Crear workspace exclusivo para `job_id`.
2. Copiar inputs verificados.
3. Registrar versiones de runner, MetaEditor y MT5.
4. Capturar estado inicial relevante.

Compilación

1. Resolver includes permitidos.
2. Ejecutar MetaEditor mediante wrapper controlado.

3. Capturar log y código de retorno.
4. Aplicar política de errores/warnings.
5. Calcular hash del `.ex5`.

Smoke test y backtest

1. Verificar inicialización.
2. Ejecutar escenario solicitado.
3. Controlar duración y recursos.
4. Capturar logs y reporte.
5. No cambiar preset ni ejecutable dentro del mismo job.

Incubación

Cuando el job incluya incubación:

1. Crear/registrar deployment local.
2. Verificar Magic Number, símbolo, timeframe y slot.
3. Solicitar aprobación local de inicio.
4. Iniciar y registrar timestamp.
5. Monitorizar logs, uptime e incidentes.
6. Permitir status y stop controlados.
7. Solicitar aprobación cuando la política lo requiera.

15. Workflow F — Crear y devolver evidencia

F1. Recopilar

- job original o su hash;
- acknowledgements;
- versiones efectivas;
- log de compilación;
- hash del `.ex5`;
- preset efectivo y hash;
- reporte MT5;
- tester/terminal logs permitidos;
- eventos de incubación;
- incidentes y excepciones.

F2. Crear manifest

```
schema_version: 1
evidence_bundle_id: evb_01...
job_id: job_01...
runner_release: 1.0.0
runner_id: mt5-vps-01

execution:
  status: completed
  operator_alias: partner-01
  started_at: "..."
  completed_at: "..."

toolchain:
  metaeditor_build: "..."
  mt5_build: "..."

outputs:
  executable_sha256: "..."
  effective_preset_sha256: "..."
  report_sha256: "..."
```

F3. Publicar

1. Validar el bundle localmente.
2. Crear ZIP y hash final.
3. Copiarlo a `mt5-to-tradinglab/evidence/`.
4. Emitir acknowledgement `completed` o `failed`.
5. Conservar copia local según retención.

16. Workflow G — Ingerir y reconciliar evidencia

G1. Ingreso seguro

1. Copiar desde Drive a `_storage/inbox/`.
2. No abrir ni ejecutar el contenido directamente.
3. Validar límites, rutas, esquema y hashes.
4. Enviar inconsistencias a `_storage/quarantine/`.

G2. Persistencia

1. Conservar el bundle original en storage.
2. Registrar artefactos y hash del paquete.
3. Relacionarlo con `job_id`, build y entorno.

G3. Reconciliación

Comparar solicitado frente a ejecutado:

- runner esperado/real;
- fuente y preset;
- MetaEditor/MT5;
- `.ex5` esperado/real;

- símbolo y timeframe;
- periodo y condiciones;
- acciones requeridas/completadas;
- outputs obligatorios.

G4. Nivel de evidencia

- `verified`: manifiestos, hashes y campos obligatorios consistentes.
- `partially_verified`: evidencia válida pero incompleta.
- `self_reported`: declaración sin verificación técnica suficiente.
- `rejected`: corrupta, inconsistente, expirada o no relacionada.

G5. Importación

1. Parsear sin modificar el original.
2. Importar métricas idempotentemente.
3. Relacionar run, deployment y evidence bundle.
4. Registrar diferencias SQX/MT5.
5. Cerrar o invalidar el job/run.

17. Automatización progresiva

Nivel 0 — Manual documentado

- Paquetes creados con scripts asistidos.
- Transferencia manual por Drive.
- Verificación manual de hash.
- Aprobación humana en todas las etapas.

Nivel 1 — Runner CLI

- Instalador y self-test.
- Importación y validación automática de paquetes.
- Wrappers de compilación/backtest.
- Evidence bundle automático.
- Aprobación humana conservada.

Nivel 2 — Polling controlado

- El runner consulta periódicamente carpetas asignadas.
- Descarga jobs a inbox.
- Valida automáticamente.
- Nunca ejecuta solo por detección; crea una solicitud local pendiente.

Nivel 3 — Políticas de autoaprobación limitadas

- Solo jobs de bajo riesgo y esquemas conocidos.
- Allowlist estricta.
- Límites de recursos.

- Kill switch local.
- Deployment e inicio de incubación continúan con aprobación humana salvo decisión posterior.

Nivel 4 — Release e intercambio endurecidos

- Paquetes firmados.
- Releases inmutables.
- Attestations de procedencia.
- Alertas y auditoría central mediante evidencias, sin control remoto de la VPS.

18. Seguridad del runner

Privilegios

- Usuario Windows no administrador.
- Acceso limitado a `C:\TradingLabMT5`, ubicaciones necesarias de MT5 y carpeta de intercambio.
- Sin acceso al catálogo o storage de TradingLab.
- Sin credenciales del repositorio principal inicialmente.

Ejecución

- Acciones declarativas y allowlist.
- Sin shell arbitrario proveniente del job.
- Argumentos construidos por el runner, no concatenados desde texto externo.
- Timeouts y límites de tamaño/recursos.
- Directorio exclusivo por job.
- Logs estructurados e inmutables durante el proceso.

Paquetes

- Hashes SHA-256.
- Esquemas versionados.
- Rechazo de rutas absolutas o `..`.
- Defensa ante archivos comprimidos excesivos.
- No ejecutar HTML, EXE, DLL o scripts inesperados.
- El `.ex5` se utiliza únicamente dentro del workflow aprobado.

Red

- No se abre un listener del runner hacia TradingLab.
- El intercambio es pull/manual y salida hacia Drive según política.
- Las capacidades de red de los EA se deshabilitan salvo excepción aprobada.

19. Auditoría

Cada evento incluye cuando corresponda:

- `correlation_id`;

- `runner_id` y versión;
- `release_id`;
- `job_id`;
- `strategy_version_id`;
- `bot_build_id`;
- `deployment_id`;
- `run_id`;
- operador mediante alias;
- timestamp UTC;
- acción, resultado y hash.

La falta de conectividad directa no impide la auditoría: los eventos viajan dentro de acknowledgements y evidence bundles.

20. Fallos y recuperación

Release inválido

No instalar. Mover a rejected, conservar diagnóstico y solicitar nueva versión.

Job inválido o expirado

No ejecutar. Emitir acknowledgement `rejected` con código de causa.

Fallo de compilación

Conservar log, marcar job `failed` y generar evidencia parcial.

Fallo de MT5

Detener el flujo de forma segura, recopilar logs y preservar workspace hasta decidir retención.

Drive no disponible

El runner continúa gestionando operaciones locales ya aprobadas según política, pero encola acknowledgements/evidencias. TradingLab no considera reconciliado el trabajo hasta recibirlos.

Runner comprometido o inconsistente

Detenerlo, revocar credenciales locales de Drive si existen, preservar logs, reinstalar desde versión verificada y revisar deployments antes de reanudar.

21. Estrategia Git

Decisión

No utilizar una rama permanente `mt5-server`.

Motivos

- Las ramas no limitan qué puede leer una credencial.
- Generan divergencia y omisión de parches.
- No identifican una instalación concreta.
- Mezclan desarrollo con distribución.

Estrategia adoptada

- Código fuente en `main`.
- Versiones semánticas del runner.
- Tags futuros `mt5-runner/vX.Y.Z`.
- Paquetes construidos desde commit limpio.
- Changelog y manifest por versión.
- Releases y rollback explícitos.

22. Futuras mejoras con GitHub

Releases inmutables

Cuando el repositorio y plan lo permitan, TradingLab podrá publicar el runner como GitHub Release inmutable. Esto impide modificar después los assets y el tag asociados a la versión. La VPS puede verificar que el archivo descargado coincide exactamente con el asset publicado mediante `gh release verify-asset`. Véase la documentación oficial de verificación de releases.

Artifact attestations

GitHub Actions puede generar attestations criptográficas que relacionan un paquete con repositorio, commit y workflow de construcción. Deben verificarse para aportar valor y no garantizan por sí solas que el software sea seguro. Véase la documentación oficial de artifact attestations.

Posibles modelos

Modelo A — GitHub produce; Drive transporta

1. GitHub construye release y attestation.
2. TradingLab verifica y copia el asset aprobado a Drive.
3. La VPS MT5 no necesita credenciales GitHub.
4. El operador verifica hash y, si se instala GitHub CLI, también procedencia.

Es la evolución más compatible con el aislamiento actual.

Modelo B — Descarga directa desde repositorio de distribución

1. Un repositorio privado separado contiene únicamente releases del runner.
2. La VPS usa una credencial de solo lectura limitada a ese repositorio.
3. Descarga una versión fijada, nunca `latest` sin validación.
4. Verifica release, asset y attestation.

No se otorgaría acceso al monorepo principal. Si se utilizan deploy keys, serán de solo lectura; GitHub advierte que una deploy key con escritura tiene privilegios equivalentes a colaboradores con capacidad de escritura. Véase la

documentación oficial de deploy keys.

Sparse checkout

No se considerará una frontera de seguridad. `git sparse-checkout` limita qué archivos aparecen en el working tree, pero no restringe la capacidad de lectura de la credencial sobre el repositorio. Puede ser útil por comodidad, no para proteger el resto de TradingLab. Véase la documentación oficial de Git sparse-checkout.

Condiciones para migrar

- Política GitHub aprobada.
- Repositorio y visibilidad definidos.
- Verificación probada en Windows/VPS.
- Procedimiento de rotación/revocación de credenciales.
- Rollback probado.
- No ampliar la conectividad entrante.
- Aprobación de los socios.

23. Criterios de aceptación del workflow

1. Construir un runner desde commit limpio.
2. Generar manifest y checksums reproducibles.
3. Transferirlo sin acceso remoto a la VPS MT5.
4. Verificar e instalar una versión concreta.
5. Ejecutar self-test y revertir a la versión anterior.
6. Crear y validar un job sin comandos arbitrarios.
7. Requerir aprobación local para acciones sensibles.
8. Compilar o ejecutar MT5 mediante wrappers controlados.
9. Generar acknowledgement y evidence bundle completos.
10. Ingerir evidencia sin procesarla directamente desde Drive.
11. Reconciliar solicitado frente a ejecutado.
12. Operar temporalmente si Drive está caído sin perder evidencia.
13. No exponer secretos ni acceso entrante.
14. Identificar runner, job, build, deployment y run de extremo a extremo.

24. Decisiones pendientes

1. Runtime y lenguaje del runner.
2. Usuario Windows y permisos exactos.
3. Método local para almacenar credenciales de Drive.
4. Política de firma de scripts y paquetes.
5. Acciones con aprobación obligatoria.
6. Intervalo de polling futuro.
7. Tamaño máximo y formatos admitidos.
8. Retención local de workspaces y evidencia.
9. Estrategia de actualización de MetaEditor/MT5.

10. Requisitos para habilitar releases inmutables de GitHub.

25. Conclusión

TradingLab MT5 Runner permite automatizar el ciclo completo sin convertir la VPS MT5 en una extensión administrable desde TradingLab. La seguridad se mantiene porque la ejecución ocurre localmente, con paquetes declarativos, verificación, aprobación y evidencia. Google Drive resuelve el intercambio inicial; releases inmutables y attestations de GitHub podrán reforzar después la procedencia sin alterar el principio de aislamiento.